

Introduction to Programming with R

Basic Syntax and Operators

Selene R. Schmittling

2025-08-01

Learning Outcomes

In this Session of our R Workshop, we will cover the following introductory aspects of R Studio & Programming:

- R vs. RStudio
- RStudio Environment
- Notebooks & Scripts
- Variables & Data Types
- Anatomy of Base R Functions
- Data Structures
- Operators

By the end of this Session, you will understand:

1. The difference between R and RStudio
2. The default layout of the RStudio Environment
3. What a variable is and how to declare one
4. Basic anatomy of a function in R
5. Basic data types available in R
6. Basic data structures and how to access the data in them
7. What an operator is and some available in R

R vs. RStudio: What's the Difference?

- R is an open-source programming language
 - used widely for data science and statistical analysis.
- Open-source means free to use and:
 - Access to many libraries and packages
 - Some tested more diligently than others
- RStudio is an programming environment
 - makes it easier to write and test scripts developed in R.

RStudio Environment

There are four main window panes in RStudio: the Source, the Console, the Environments and the Output Win-

The screenshot displays the RStudio interface with four main panes:

- Source Pane:** Contains an R script named `ggplot2.R` with the following code:

```
1 library(ggplot2)~
2 mpg_plot <- ggplot(mpg, aes(x = displ, y = hwy)) +~
3   geom_point(aes(colour = class))~
4 ~
5 mpg_plot|
6 |
```
- Console Pane:** Shows the execution of the code from the Source pane:

```
> library(ggplot2)
> mpg_plot <- ggplot(mpg, aes(x = displ, y = hwy)) +
+   geom_point(aes(colour = class))
>
> mpg_plot
> |
```
- Environments Pane:** Displays the Global Environment with a table showing the `mpg_plot` object:

Name	Type	Len...	Size	Value
mpg_plot	gg	9	29.1...	List of 9
- Output Pane:** Displays a scatter plot of `hwy` vs `displ` colored by `class`. The legend indicates the following classes: 2seater, compact, midsize, minivan, pickup, subcompact, and suv.

dow.

Source Pane

- Shows open scripts, Notebooks and other files you are editing.
- The area where you can view data

We'll talk about scripts and Notebooks soon.

Console Pane

- Features 3 tabs by default:
 - Console Tab
 - * Enter commands that run immediately
 - * Good for testing
 - Terminal
 - * let's you interact with your computer terminal
 - Background Jobs
 - * provides information on “background” scripts which run in a separate dedicated R session.

Environments Pane

The Environments pane has four tabs available by default:

- Environment
 - Provides info about variables defined in during your session
- History
 - Shows all the commands you have run during your session
 - Can be saved across sessions
 - Can be copied to console or a script
- Connections
 - beyond the scope of this workshop
 - check out this info for more information
- Tutorial
 - provides access to interactive tutorials.
 - Check them out!

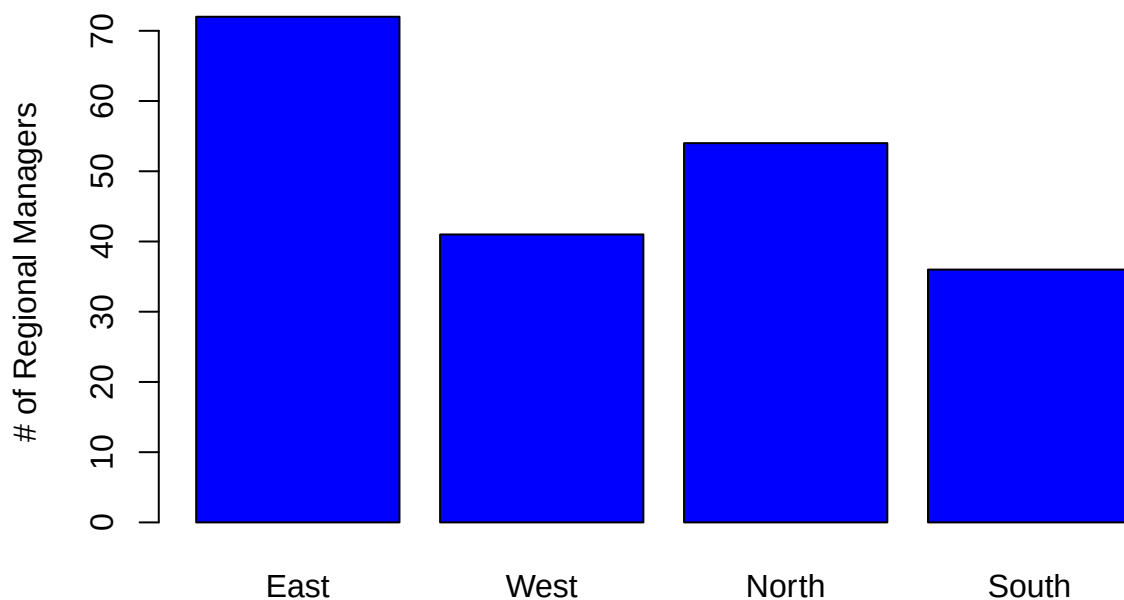
Output

The output pane has a variety of tabs that provide access to different types of output. This includes files, plots, packages, help and a couple of other specialized output types. Tabs in this pane include:

- Files
 - shows your file directory
 - projects created in a directory will set the “working directory” to the directory it is stored in
 - without a project, you may need to set a “working directory”
 - setting a “working directory” lets you use relative path names
- Plots
 - this is where plots will show up when you generate them

```
# Lets create a little plot
# create some data
data <- data.frame(a = c(72,41,54,36), b=c('East','West','North','South'))

# generate plot
barplot(data$a,
        names.arg = data$b,
        col="blue",
        ylab="# of Regional Managers")
```



```
# function name
# function parameters
# return values
```

- Packages
 - Interface to install packages
 - Installed packages
 - * Any package listed
 - Loaded packages
 - * Any package with a checkmark to the left of its name
 - Help
 - Your best friend!!
 - You can see the details about functions you want to use
 - use search bar in pane or type “?barplot” in console
 - Viewer
 - Shows output for rendered notebooks
 - Presentation
 - Shows output when you generate slides using Quarto
-

Quiz # 1

Where do you look for...?

1. Where would you look for the value of a variable you set?
 - Source Pane
 - Terminal Tab in the Console Pane
 - Environment tab in the Environment Pane
 - Viewer tab in the Output Pane
2. Where would you find the output of a plot command?
 - Plots tab of the Output pane
 - Source pane
 - History tab of the Environment pane
 - In a notebook, below the code block that generates the plot
3. What pane is your script or notebook file open in?
 - Source pane
 - Console pane
 - Environment pane
 - Output pane

Quiz # 1 ANSWERS

1. Environment tab in the Environment Pane
 2. Plots tab of the Output pane
 3. Source Pane
-

Notebooks & Scripts

Through RStudio, you can use and render R code in a variety of ways including:

- Scripts
 - These are the way most people start out.
 - A script has a file type of .R
 - You can easily run the entire script using the Run command
 - Only comments and code.
- Notebooks
 - Great way to have text and code together
 - Can contain text, code chunks, equations, tables
 - Good when small bits of code are being run for a discrete task.

Even though you will probably use scripts when you start out coding, for this workshop we are going to work in a notebook to take advantage of the ability to have readable text along side runnable code.

Variables in R

Variables are named containers used to store information. What can we name them? Almost anything we want, according to the following rules:

- name has to start with a letter
- can contain any combination of letters, numbers and can include periods “.”
- can’t start with an underscore, “_”
- are case sensitive
- can’t be any of the reserved words in R

What R reserved words?

All programming languages have reserved words. These are words that have a specific meaning to the language itself. Some examples in R include:

- TRUE
- FALSE
- NULL
- if

You can find a list of reserved words [here](#).

Will it Variable?

For a variable to be useful, you must give it a value. To set a variable to a value, you use the assignment operator. You’ve seen this in math, for instance, $a + b = 5$. In R, to assign a value to a variable we use “<-”. We’ll talk more about operators later, but now you know the assignment operator!

The code box below shows some example variables. Use CMD/CTRL + RETURN/ENTER to run a line of code.

```
bert <- 5
ernie <- "Bert's Best Friend"
mrs.wuff <- 9.732
TuFfY <- "8/18/2025"
tuffy_2 <- as.POSIXct("8/18/2025", format="%m/%d/%Y")
i.am.happy = TRUE # Wait, what?!?! what's with the "="?

# See the variable value
bert

## [1] 5
```

```
ernie
```

```
## [1] "Bert's Best Friend"
```

I Digress ...

In most programming languages, you use “=” as the assignment operator, like you see with `i.am.happy` above. While R lets you use “=”, the convention is to use “<-”. This is a holdover from the language from which R is derived, S. Programmers are funny, funny people.

QUIZ # 2

Using the rules listed above which of the following passwords will work?

- `My_favoritePASSWORDofAll.tymes`
- `_favoritePW`
- `PW6`
- `PW6_myfav`
- `6PW`
- `If`
- `if`

```
# Not sure which will run?
```

```
# Use the assignment operator to assign the value 5 to a variable using the names above. Feel free to t
```

```
My_favoritePASSWORDofAll.tymes <- 5
```

Quiz # 2 Answers

The acceptable passwords are:

- `My_favoritePASSWORDofAll.tymes`
 - `PW6`
 - `PW6_myfav`
 - `If`
-

Data Types

Data Types

- are categories of data
- dictate
 - how much space a variable of that type will take
 - determine how they are handled as input and output
 - how you can use the variable
- sometimes R cares about what data type you use
- sometimes R doesn’t care (e.g., is really flexible)

We’ll talk about 4 main data types:

- Numeric
- Text
- Date/Time
- Logical or Boolean

Numeric

Numeric values can be described as integers, decimals (floating point numbers) and complex. We will focus on integers and decimals, but be aware that R can handle complex values too.

Some languages aggressively distinguish between integers (whole numbers) and floating point (decimal) values. R does not. In R, when you store a number it is loaded as a **double**. This is just another word for decimal or floating point.

In most situations, you don't have to worry about whether a number is an integer or a floating point value. However, it does recognize integers and doubles (floating point). It also has functions that check or convert a number to integers and floats. Read more about numbers in R [here](#).

Let's look at some examples.

```
# Let's set some variables to an integer.
myBelovedInteger <- -53
BigInteger <- 1500225
typeof(BigInteger)

## [1] "double"

# Uncomment the line below.
# Look for a red x to the left of the line number
# Try to run the line of code.

# NoCommasReally <- -22,550,675

# Remember to re-comment the line because the error may keep the notebook from rendering.
```

REMEMBER: computers don't need commas in numbers. That's just to make them easier to read for us humans.

```
# Now we'll define some floating point variables
myPreciousFloat <- 42.5 # Even floating point numbers need love
RandomFloatIdontKnow <- 1999999999.999999999999999
swimFloaties <- -25.11235813

# stillNoCommas <- 4,325.98

trickyFloat <- 198.0 # is it an integer or is it a double?
typeof(trickyFloat) # everything is a double to R

## [1] "double"

typeof(myPreciousFloat) # yup, still a double

## [1] "double"
```

Use the `typeof()` function to check what data type R thinks a variable is.

Text

We've looked at the scary, scary numbers. Let's look at good old familiar text. Run the code in the block below. What data type does R see string data as? Hint: run the last line!!

```
# Some text examples
myFirstString <- "String"
mySecondString <- "My second string"
```

```

anotherString <- "Is this a string or a sentence? I'm not sure..."

ughWhatNow <- 'She asked, "What if I want to put something in quotes?"'
print(ughWhatNow)

## [1] "She asked, \"What if I want to put something in quotes?\""
typeof(ughWhatNow)

## [1] "character"

ughWhatNowPart2 <- "She asked, 'What if I want to put something in single quotes?'"
print(ughWhatNowPart2)

## [1] "She asked, 'What if I want to put something in single quotes?'"

```

Compare how `ughWhatNow` is printed to how `ughWhatNowPart2` is printed. What is the difference? Where did the two “\” come from?

First, the “\” is called an escape character. You use it to differentiate between a character that should be used as a character as opposed to indicating the beginning or end of a string. You might also use it to indicate a non-printing character. An example would be “\n” which is a line feed. In this case, we want to have a string that includes quoted text, but we use quotes to tell R something is a string. The example above shows that R uses double quotes to identify the beginning and end of a string and if we need to include quoted text, we should use single quotes.

Dates & Times

Dates are their own special kind of special. The Date/Time data type is useful for:

- Outputting dates so they look pretty and conform to the style you want them to (e.g., 8/26/2025, August 26, 2025; 25/08/2025).
- Being able to do date math: how many weeks, days, months, years between two dates, etc.

We can define a date as a string. The computer doesn’t care:

```

aDateButItsReallyAString <- "07/15/2025"
print(aDateButItsReallyAString)

```

```
## [1] "07/15/2025"
```

Why would we want to use a date data type? See above or try the code below. Uncomment the line that states with `letsAdd4Days` and try to run it.

```

# To explore when data strings break, uncomment the next line and try to run it. We are just trying to .

# letsAdd4Days <- aDateButItsReallyAString + 4

# !! Recoment it when you are done. Otherwise you notebook won't render.

```

Using text dates has it’s limitations. What if you didn’t want to print it out as “07/15/2025”, but wanted to show “July 15, 2025”?

That’s where the date data type comes in.

```

# Let's try some dates
myTextDate = "7/15/25"
print(myTextDate)

```

```
## [1] "7/15/25"
```



```
myDateDate = as.Date(myTextDate, format = "%m/%d/%y")
print(myDateDate)
```

```
## [1] "2025-07-15"
```

```
myDateDate2 = as.Date(myTextDate, format = "%m/%d/%Y")
print(myDateDate2)
```

```
## [1] "0025-07-15"
```

What is with the output on the third line? Good question! Look at the format string passed in the `as.Date()` function. Compare the formats used to define `myDateDate` and `myDateDate2`. What's different?

The question is can we add 4 days to `myDateDate` now?

```
myNewDateDate <- myDateDate + 4
myNewDateDate
```

```
## [1] "2025-07-19"
```

Yes!

Logical

A Boolean value is one that can take one of two values: 0 or 1. In R, these types of values are represented using the “logical” data type. The logical data type takes the value TRUE (1) or FALSE (0). It is good for setting flags and creating indices to subset data.



Figure 1: Examples for logical values

```
# Example of boolean
dogIsCute <- TRUE
glassIsFull <- TRUE
```

```
todoIsDone <- FALSE
doorIsClosed <- FALSE
```

When we get to the section on operators, you will begin to see the extent to which we use logical values in programming. It make sense when you remember that computers only understand voltage off/voltage on which gets encoded as 0/1. So, of course we would use logical values a lot. It is the most basic thing computers can manipulate.

Quiz # 3

```
# Create a variable and set it to some text.
sailBoatType <- "Sonar"

# Create a variable and set it to a numeric value
boatLength <- 23 # Length in Feet

# Create a text variable and set it to a text date.
sailDateText <- "09-15-1997"

# Create a date variable and convert the text date to a date using `as.Date()`
sailDate <- as.Date(sailDateText, format="%m-%d-%Y")

# Print your date variable to the screen
print(sailDate)

## [1] "1997-09-15"
```

Data Structures

So, we've looked at the basic data types in R. In this section, we'll explore ways values can be stored in R. When we discuss how data is stored in a programming language we refer to data structures.

We will explore three widely used data structures:

- arrays/vectors
- matrices
- dataframes

Basic distinguishing factors between data structions include:

- do they store only one data type or multiple data types?
- do they have one dimension, two dimensions or > 2 dimensions?

There are two aspects of using data structures:

- how to create them; and,
- how to access the data in them.

We'll cover both.

Vectors (1-D Arrays)

Vectors store data in one dimension and allow only one data type to be stored.

I Digress ...

In point of fact, R is a vectorized language. What does that mean? It means scalars (single values) in R are really 1-element vectors. Will this have a big impact on learning R? No, probably not. But you can whip this fact out at parties and amaze your friends.

How to Create Vectors

Creating vectors is shown in the code below. We'll make one for integers, floating point values, text and Boolean. No dates.

```
# Let's clean up the variables to make it easier to see what's going on
rm(list=ls())
```

```
# EXAMPLES OF HOW TO CREATE VECTORS
# (Single Data Type)
```

```
# Integer. Ok, fine, double.
intVector = c(1, 2, 3, 4, 5)
print(intVector)
```

```
## [1] 1 2 3 4 5
```

```
# Decimal. Ok, fine, double.
floatVector = c(1.00, 2.25, 3.50, 4.75, 5.00)
print(floatVector)
```

```
## [1] 1.00 2.25 3.50 4.75 5.00
```

```
# Text
txtVector = c("one", "two", "three", "four", "five")
print(txtVector)
```

```
## [1] "one" "two" "three" "four" "five"
```

```
# Boolean/Logical
logiVector = c(TRUE, FALSE, FALSE, TRUE, TRUE)
print(logiVector)
```

```
## [1] TRUE FALSE FALSE TRUE TRUE
```

```
# Into the weeds we go ....
```

```
# Also a vector:
```

```
dohVector <- c(1, 2, 3.00, "four", "five") # Wait! What?!?!
print(dohVector)
```

```
## [1] "1" "2" "3" "four" "five"
```

I said arrays will hold only one data type and then I went and loaded one up with all sorts of data types. This was to prove a point. Just because you CAN do a thing doesn't mean you should. As you can see R helpfully converts every value to a string. This is great if this is what you wanted and expected to happen. However, if not, it could break your code.

Accessing Elements in a Vector

Elements in a vector can be accessed by using an index, or range of indices between two square brackets. Let's try it out.

```

# Make sure you ran the code above to define the vectors. Click on the green run button in the code chunk
# VECTORS
print(intVector)

## [1] 1 2 3 4 5
intVector[1]

## [1] 1
print(floatVector)

## [1] 1.00 2.25 3.50 4.75 5.00
floatVector[2]

## [1] 2.25
print(txtVector)

## [1] "one" "two" "three" "four" "five"
txtVector[3:4] # we can pull more than 1 element

## [1] "three" "four"
print(logiVector)

## [1] TRUE FALSE FALSE TRUE TRUE
logiVector[5]

## [1] TRUE
print(dohVector)

## [1] "1" "2" "3" "four" "five"
dohVector[6] # Why did we get NA? Hmm...How long is the dohVector?

## [1] NA
length(dohVector) # Oops

## [1] 5
# Let's Get Fancy!
newVector2Showoff <- c(7, 8, 9, 10, 11, 12)
indices2grab <- c(2,4,6)
newVector2Showoff[indices2grab] # We can use a vector to extract values from a vector

## [1] 8 10 12

```

Matrices (or n-D Arrays)

Matrices:

- can have 2 or more dimensions
- hold only one data type

They are useful in computational work. There are a couple of ways to declare a matrix in R.

If you only need a 2-dimensional matrix (rows and columns only), you can use `matrix()`. You define the number of rows and columns. Let's see an example below:

```
# Create a 2-D matrix
myMatrix <- matrix(c(1:8), nrow=4, ncol=2, byrow = FALSE)
print(myMatrix)
```

```
##      [,1] [,2]
## [1,]    1    5
## [2,]    2    6
## [3,]    3    7
## [4,]    4    8
```

```
# What happens if you create a matrix with more elements than your data?
myBiggerMatrix <- matrix(c(1:8), nrow=5, ncol=2, byrow=FALSE)
```

```
## Warning in matrix(c(1:8), nrow = 5, ncol = 2, byrow = FALSE): data length [8]
## is not a sub-multiple or multiple of the number of rows [5]
```

```
print(myBiggerMatrix)
```

```
##      [,1] [,2]
## [1,]    1    6
## [2,]    2    7
## [3,]    3    8
## [4,]    4    1
## [5,]    5    2
```

```
# What happens if you create a matrix with fewer elements than your data?
mySmallerMatrix <- matrix(c(1:8), nrow=3, ncol=2, byrow=FALSE)
```

```
## Warning in matrix(c(1:8), nrow = 3, ncol = 2, byrow = FALSE): data length [8]
## is not a sub-multiple or multiple of the number of rows [3]
```

```
print(mySmallerMatrix)
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
# Create a 2-D matrix filled with zeros
# that you can fill it in later.
myZeroMatrix <- matrix(0, nrow=5, ncol=6)
print(myZeroMatrix)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0
## [5,]    0    0    0    0    0    0
```

```
# Ooh let's set an element to some other value...
myZeroMatrix[4,4] <- -16
print(myZeroMatrix)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    0    0    0    0    0    0
```

```
## [2,] 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0
## [4,] 0 0 0 -16 0 0
## [5,] 0 0 0 0 0 0
```

To create a matrix that has more than 2 dimensions, you use the `array()` function. Let's give it a try.

```
# Create a 3-D matrix
my_array <- array(0, dim = c(5,10,3))
print(my_array)
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 2
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 3
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
```

```
# Set a value
my_array[1,2,3] <- 42 # [row, col, dim]
print(my_array)
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 2
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
```

```
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 3
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 42 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
```

```
print(my_array[1,2,3])
```

```
## [1] 42
```

```
# Set the value of the 5th column, 3rd row and 2nd dimension of myArray to your favorite number. Print
```

```
my_array[3,5,2] <- 7892
```

```
print(my_array)
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 2
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 7892 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
##
## , , 3
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 0 42 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0 0
```

Dataframes (Tabular Data)

Dataframes:

- are 2-dimensional tabular structures for storing data (rows and columns)
- allow multiple datatypes, but each column must be the same datatype.
- are similar to excel tables.

dataframes can have row and column names.

```

# Create a dataframe
myDataFrame <- data.frame(Name = c("Snoopy", "Scrat", "Ratbert", "Hobbs"),
                          Species = c("Dog", "Squirrel", "Rat", "Tiger"),
                          numFriends = c(8, 0, 1, 1))

print(myDataFrame)

##      Name Species numFriends
## 1 Snoopy    Dog         8
## 2  Scrat Squirrel         0
## 3 Ratbert   Rat         1
## 4  Hobbs   Tiger         1

# extract data from dataframe
myDataFrame["Name"] # extracts the values in the Name column

##      Name
## 1 Snoopy
## 2  Scrat
## 3 Ratbert
## 4  Hobbs

myDataFrame[c("Name", "numFriends")] # extract values for Name and numFriends columns

##      Name numFriends
## 1 Snoopy         8
## 2  Scrat         0
## 3 Ratbert         1
## 4  Hobbs         1

myDataFrame[1:2] # extracts the first two columns

##      Name Species
## 1 Snoopy    Dog
## 2  Scrat Squirrel
## 3 Ratbert   Rat
## 4  Hobbs   Tiger

myDataFrame[,1:2] # also extracts the first two columns

##      Name Species
## 1 Snoopy    Dog
## 2  Scrat Squirrel
## 3 Ratbert   Rat
## 4  Hobbs   Tiger

myDataFrame[1:2, ] # extracts the first two rows

##      Name Species numFriends
## 1 Snoopy    Dog         8
## 2  Scrat Squirrel         0

myDataFrame[-1] # extracts all columns but the first column

##      Species numFriends
## 1      Dog         8
## 2 Squirrel         0
## 3      Rat         1
## 4      Tiger        1

```



```
myDataFrame[-1,] # extracts every row but the first one
```

```
##      Name Species numFriends
## 2   Scrat Squirrel         0
## 3 Ratbert    Rat         1
## 4   Hobbs   Tiger         1
```

Quiz # 4

```
# Create some data to use for the problems below. Make sure you run this line of code
data <- c(1:24)
```

```
# Create an array/vector that holds 4 words
myFavColors <- c("green", "purple", "black", "cyan")
```

```
# Create a two-dimensional matrix and fill it with `data`
# Try using dimensions that fit the data and dimensions that don't
```

```
# Define matrix to be the same number of elements as data
myMatrix1 <- matrix(data, nrow=6, ncol=4)
print(myMatrix1)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    7   13   19
## [2,]    2    8   14   20
## [3,]    3    9   15   21
## [4,]    4   10   16   22
## [5,]    5   11   17   23
## [6,]    6   12   18   24
```

```
# Define matrix to be larger than data
myMatrix2 <- matrix(data, nrow=6, ncol=5)
```

```
## Warning in matrix(data, nrow = 6, ncol = 5): data length [24] is not a
## sub-multiple or multiple of the number of columns [5]
```

```
print(myMatrix2)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    7   13   19    1
## [2,]    2    8   14   20    2
## [3,]    3    9   15   21    3
## [4,]    4   10   16   22    4
## [5,]    5   11   17   23    5
## [6,]    6   12   18   24    6
```

```
# Define matrix to be smaller than data
myMatrix3 <- matrix(data, nrow=5, ncol=4)
```

```
## Warning in matrix(data, nrow = 5, ncol = 4): data length [24] is not a
## sub-multiple or multiple of the number of rows [5]
```

```
print(myMatrix3)
```

```
##      [,1] [,2] [,3] [,4]
```

```
## [1,] 1 6 11 16
## [2,] 2 7 12 17
## [3,] 3 8 13 18
## [4,] 4 9 14 19
## [5,] 5 10 15 20
```

```
# Create a 3-dimensional matrix and fill it with `data`
# Try using dimensions that fit the data and dimensions that don't
my3DMatrix <- array(data, dim = c(3,2,4))
print(my3DMatrix)
```

```
## , , 1
##
##      [,1] [,2]
## [1,] 1 4
## [2,] 2 5
## [3,] 3 6
##
## , , 2
##
##      [,1] [,2]
## [1,] 7 10
## [2,] 8 11
## [3,] 9 12
##
## , , 3
##
##      [,1] [,2]
## [1,] 13 16
## [2,] 14 17
## [3,] 15 18
##
## , , 4
##
##      [,1] [,2]
## [1,] 19 22
## [2,] 20 23
## [3,] 21 24
```

```
# Make a 3-dimensional matrix and fill it with zeros
# pick whatever dimensions you want, but don't break your computer with huge values!
# print out matrix
myZeros <- array(0, dim = c(9,9,9))
print(myZeros)
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0
## [6,] 0 0 0 0 0 0 0 0 0
## [7,] 0 0 0 0 0 0 0 0 0
```

```

## [8,] 0 0 0 0 0 0 0 0 0
## [9,] 0 0 0 0 0 0 0 0 0
##
## , , 2
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0
## [6,] 0 0 0 0 0 0 0 0 0
## [7,] 0 0 0 0 0 0 0 0 0
## [8,] 0 0 0 0 0 0 0 0 0
## [9,] 0 0 0 0 0 0 0 0 0
##
## , , 3
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0
## [6,] 0 0 0 0 0 0 0 0 0
## [7,] 0 0 0 0 0 0 0 0 0
## [8,] 0 0 0 0 0 0 0 0 0
## [9,] 0 0 0 0 0 0 0 0 0
##
## , , 4
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0
## [6,] 0 0 0 0 0 0 0 0 0
## [7,] 0 0 0 0 0 0 0 0 0
## [8,] 0 0 0 0 0 0 0 0 0
## [9,] 0 0 0 0 0 0 0 0 0
##
## , , 5
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,] 0 0 0 0 0 0 0 0 0
## [2,] 0 0 0 0 0 0 0 0 0
## [3,] 0 0 0 0 0 0 0 0 0
## [4,] 0 0 0 0 0 0 0 0 0
## [5,] 0 0 0 0 0 0 0 0 0
## [6,] 0 0 0 0 0 0 0 0 0
## [7,] 0 0 0 0 0 0 0 0 0
## [8,] 0 0 0 0 0 0 0 0 0
## [9,] 0 0 0 0 0 0 0 0 0

```

```

##
## , , 6
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,]    0    0    0    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0    0    0    0
## [5,]    0    0    0    0    0    0    0    0    0
## [6,]    0    0    0    0    0    0    0    0    0
## [7,]    0    0    0    0    0    0    0    0    0
## [8,]    0    0    0    0    0    0    0    0    0
## [9,]    0    0    0    0    0    0    0    0    0
##
## , , 7
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,]    0    0    0    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0    0    0    0
## [5,]    0    0    0    0    0    0    0    0    0
## [6,]    0    0    0    0    0    0    0    0    0
## [7,]    0    0    0    0    0    0    0    0    0
## [8,]    0    0    0    0    0    0    0    0    0
## [9,]    0    0    0    0    0    0    0    0    0
##
## , , 8
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,]    0    0    0    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0    0    0    0
## [5,]    0    0    0    0    0    0    0    0    0
## [6,]    0    0    0    0    0    0    0    0    0
## [7,]    0    0    0    0    0    0    0    0    0
## [8,]    0    0    0    0    0    0    0    0    0
## [9,]    0    0    0    0    0    0    0    0    0
##
## , , 9
##
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## [1,]    0    0    0    0    0    0    0    0    0
## [2,]    0    0    0    0    0    0    0    0    0
## [3,]    0    0    0    0    0    0    0    0    0
## [4,]    0    0    0    0    0    0    0    0    0
## [5,]    0    0    0    0    0    0    0    0    0
## [6,]    0    0    0    0    0    0    0    0    0
## [7,]    0    0    0    0    0    0    0    0    0
## [8,]    0    0    0    0    0    0    0    0    0
## [9,]    0    0    0    0    0    0    0    0    0

```

```

# Make a dataframe with two rows and three columns of data
# Print out the dataframe
myDataframe <- data.frame(id = 1:2,
                          color = c("green", "blue"),
                          size = c(36, 92))

print(myDataframe)

##   id color size
## 1  1 green   36
## 2  2 blue   92

```

Operators

Operators are used to perform ... well, ... operations. They provide a way to compare values and variables. Operators may sound exotic, but you've encountered them in any basic math class you've ever taken. We've covered the assignment operator... quick what is the assignment operator?

We'll cover the following kinds of operators:

- Arithmetic,
- Relational, and
- Logical.

Arithmetic

I'm sure you can already guess what these are:

Operator	Unary / Binary	Operation	Example
-	U	negation	-5
+	B	addition	amt1 + amt2
-	B	subtraction	amt1 - amt2
*	B	multiplication	2*amt1
/	B	division	amt1/2
^ or **	B	power	2^3
%%	B	modulus	10 %% 3
%/%	B	integer division	10 %/% 3

Operators can be referred to as **unary** or **binary**. From math class, you may recall that operators act upon **operands**. Unary means that an operator takes a single operand while binary indicates it takes two. The - operator is a great example. It is not essential you know this, but if you hear it in the future you'll know what it means.

```

# Let's clear out our variables
rm(list=ls())

# Unary
aNumber = 5
myNegativeNumber <- -aNumber
print(myNegativeNumber)

## [1] -5

```

```
# Binary
x <- 5
y <- 10
myDifference <- x-y
print(myDifference)
```

```
## [1] -5
```

Most of the arithmetic operators should be familiar. Examples are provided below:

```
# Use operators with values
5 + 6
```

```
## [1] 11
```

```
5 - 6
```

```
## [1] -1
```

```
5 * 6
```

```
## [1] 30
```

```
5 / 6
```

```
## [1] 0.8333333
```

```
5 ^ 6
```

```
## [1] 15625
```

```
5 ** 6
```

```
## [1] 15625
```

```
# Separate Output
print('* * * * *')
```

```
## [1] "* * * * *"
```

```
# Use familiar operators with variables
# We'll be creative and use new values
```

```
# Declare some variables and set them to values
```

```
x <- 7.985
```

```
y <- 2.001
```

```
# Apply operators
```

```
x + y
```

```
## [1] 9.986
```

```
x - y
```

```
## [1] 5.984
```

```
x * y
```

```
## [1] 15.97799
```

```
x / y
```

```
## [1] 3.990505
```

```
x ^ y
```

```
## [1] 63.89283
```

```
x ** y
```

```
## [1] 63.89283
```

However, some operators may be new. These include the modulus operator, `%%`, and the special operator for integer division, `%/%`. These two operators provide special functionality. The modulus operator returns the remainder of integer division. The operator to perform integer division, returns the quotient of the division.

```
myNumber <- 10
```

```
myFirstDivisor <- 2 # divides evenly
```

```
myOtherDivisor <- 3 # doesn't divide evenly
```

```
# Integer Division & Modulus using first divisor
```

```
myNumber %/% myFirstDivisor
```

```
## [1] 5
```

```
myNumber %% myFirstDivisor
```

```
## [1] 0
```

```
# Separate Output
```

```
print("* * * * *")
```

```
## [1] "* * * * *
```

```
# Integer Division & Modulus using second divisor
```

```
myNumber %/% myOtherDivisor
```

```
## [1] 3
```

```
myNumber %% myOtherDivisor
```

```
## [1] 1
```

Relational

Relational operators are all binary and are used to compare two values. The result of using relational operators is a logical (Boolean) value: `TRUE` or `FALSE`.

Operator	Operation	Example
<code><</code>	less than	<code>mynum < 3</code>
<code><=</code>	less than or equal to	
<code>></code>	greater than	<code>mynum > 3</code>
<code>>=</code>	greater than or equal to	<code>mynum >= 3</code>
<code>==</code>	equality	<code>"a"=="b"</code>
<code>!=</code>	inequality	<code>"a"!="b"</code>

```
# Let's clear out our variables
```

```
rm(list=ls())
```

```
# Define some variables and set them to a value
```

```
bigWolf <- 135 # this is the weight of a wolf in pounds
```

```
medWolf1 <- 85 # this is an average sized wolf
```

```

medWolf2 <- 85.000000000001 # this is another average sized wolf
smallWolf <- 50 # small wolf

myGermanShepard <- 85 # weight of my dog, if I had one

# Let's compare my shepard to the different wolves

myGermanShepard < bigWolf

## [1] TRUE
myGermanShepard >= medWolf1

## [1] TRUE
myGermanShepard < medWolf2

## [1] TRUE
myGermanShepard > smallWolf

## [1] TRUE
# Let's look at `==` more closely
myGermanShepard == medWolf1 # perfectly fine, two integers

## [1] TRUE
myGermanShepard == medWolf2 # Seems good because 85.000000000001 is not equal to 85 BUT DON'T DO THIS

## [1] FALSE
# BUT what if that 85.000000000001 was the result of calculating an average weight. In this situation .

```

It is a bad idea to compare equality of floating point numbers. This is particularly true when the variables you are comparing are the result of calculation. In the example above, if 85.000000000001 was the results of an averaging calculation, it is quite likely that the fractional part is not informative. In practice you would want to find that comparison equal but your computer would not find it so. It is standard practice to use a tolerance to check for equality. This allows you to control the precision with which you check equality. See the example below:

```

a <- 0.4 + 0.5
b <- 0.9
epsilon <- 1e-9 # a very small number

(a-b) < epsilon # if the difference is less, then you assume equality.

## [1] TRUE
print(a-b)

## [1] 0

```

Logical

Logical operators are used to compare two logical values or two relational expressions that evaluate to logical values.

Operator	Operation	Example
&	Element-wise AND	c(TRUE, TRUE, FALSE) & c(FALSE, TRUE, FALSE)
&&	Logical AND	isCute & isBig
	Element-wise OR	c(TRUE, TRUE, FALSE) c(FALSE, TRUE, FALSE)
	Logical OR	isCute
!	Logical NOT	!isCute

```
# Use variables from last section
```

```
# Define some variables to compare
```

```
numWolves <- 50
```

```
numDogs <- 25
```

```
numCats <- 100
```

```
# Define some Boolean variables and set them
```

```
isParty <- FALSE
```

```
haveHotdogs <- TRUE
```

```
haveCheetos <- TRUE
```

```
# Check if numWolves is less than numDogs AND numCats is less than numDogs
```

```
(numWolves < numDogs) && (numCats < numDogs)
```

```
## [1] FALSE
```

```
# Check if numDogs is less than or equal to 25 OR numCats is less than or equal to 25
```

```
(numDogs <= 25) || (numCats <= 25)
```

```
## [1] TRUE
```

```
# Check if numDogs is less than 150 OR numCats is less than 150
```

```
(numDogs < 150) || (numCats < 150)
```

```
## [1] TRUE
```

```
# See if isParty is TRUE AND if numDogs is greater than 0 AND haveHotdogs is FALSE
```

```
# This could be a problem
```

```
isParty && (numDogs > 0) && !haveHotdogs
```

```
## [1] FALSE
```

```
# See if (numDogs is greater than zero AND haveHotdogs is TRUE) OR (numCats is greater than 0 AND haveCheetos is TRUE)
```

```
(numDogs > 0 && haveHotdogs) || (numCats > 0 && haveCheetos)
```

```
## [1] TRUE
```

```
# How could you fix the expression above to make sure that you have food available for both cats and dogs?
```

```
(numDogs > 0 && haveHotdogs) && (numCats > 0 && haveCheetos)
```

```
## [1] TRUE
```

Quiz # 5

```
# PROBLEM 1
# You need to see if you have more buns than hotdogs. Create two variables, numBuns and numHotdogs and .

# Declare `numBuns` and `numHotDogs`
numBuns = 6 # Seriously? This used to be a thing. Why was it a thing?
numHotdogs = 8

# Check if you have enough buns.
haveEnoughBuns <- numHotdogs == numBuns
# NOOOOOOOOO!!!

# PROBLEM 2

# We need to make sure we have potato chips for our guests for our Friday night party

# Declare the following variables:
#
# `perfectChipsPortion`: set it to the number of grams a perfect portion of potato chips would be
# `numGuests`: set it to the number of guests you've invited
perfectChipsPortion <- 500
numGuests <- 5

# Also declare
#
# `numBagsChips`: set to the number of bag of chips you picked up
# `gramsPerBag`: set this to the number of grams of chips per bag
numBagsChips <- 3
gramsPerBag <- 250

# Calculate the number of grams of chips you need for all your guests
# name the variable `chipsNeededInGrams`
chipsNeededInGrams <- perfectChipsPortion * numGuests

# Calculate how many of grams of chips you have
# name this variable `totalChipsInGrams`
totalChipsInGrams <- numBagsChips * gramsPerBag

# Create a Boolean variable and name it `haveEnoughChips`
# compare the number of chips you need to the number you have (in grams)
# You want `haveEnoughChips` to be TRUE if you have enough
haveEnoughChips <- totalChipsInGrams >= chipsNeededInGrams

# How many grams of chips do I need?
chipsNeededInGrams

## [1] 2500

# How many grams of chips do I have?
totalChipsInGrams

## [1] 750
```

```

# Do I have enough chips? Drum roll please...
haveEnoughChips

## [1] FALSE

# PROBLEM 3
# Use a logical operator to check whether you have enough buns for your hotdogs and enough chips for you
haveEnoughFood <- haveEnoughChips && (numHotdogs * numGuests == numBuns * numGuests)

# PROBLEM 4
# In this problem we are going to write code that let's use check whether a number is even.

# Declare a variable and set it to the number you want to check
num2Check <- 12

# Use modulus to find the remainder of dividing your number by 2.
# Then check if the remainder is zero.
# This can be done in two steps or one

# Two Steps
divBy2 <- num2Check %% 2
divBy2 == 0

## [1] TRUE

# One step
(num2Check %% 2) == 0

## [1] TRUE

```

In our next workshop, **Control Flow and Functions**, we'll learn about and practice:

- Conditional and Flow Control Statements
 - if
 - for
 - while
- Functions, including
 - built-in
 - library
 - user-defined
- Getting help
- File Input and Output

Before You Go

Take our Survey

We appreciate your input. Let us know how this Workshop went for you! We take your input seriously and appreciate the time you take to give it. Please fill out our survey.

Need Additional Help?

We are available to help you with whatever question you have about this Workshop or programming in general. If you run into snags or questions (we all do), as you start to create programs for yourself, reach

out to us. Send an email, make an appointment, or reserve a workstation! We are here to assist you to be successful in your data science journey.
